

Challenge Statement:

Multi-Agent Autonomous Dispute Resolution System

Introduction

Ride-hailing companies process thousands of dispute tickets daily — ranging from fare disputes and route deviations to property damage, safety incidents, and no-show charges.

These disputes are complex, subjective, and emotionally charged, requiring human agents to manually review GPS telemetry, chat logs, photos, payment records, and company policy before reaching a decision.

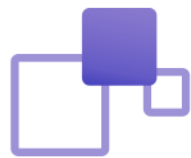
This current approach results in:

- High operational costs — millions spent annually on human support hours
- Slow resolution times — typically 24–72 hours per dispute
- Inconsistent rulings — different agents reach different conclusions on similar cases
- User churn — customers who feel unfairly treated leave the platform

Problem Statement

Build an autonomous, multi-agent dispute resolution system that can handle complex, multi-party conflicts between riders and drivers — gathering evidence, building cases, applying company policy, and issuing fair rulings quickly, transparently, and without human intervention for the majority of standard dispute categories.





What the Solution Should Solve

The AI Agent Solution — Multi-Agent Architecture

1. MVP (Required) — Core Agents

Teams must implement the following three core agents. These agents should operate on text-based and structured data evidence (GPS coordinates as data points, chat logs, fare breakdowns, timestamps).

Agent	Role	Required Capabilities
Rider Advocate Agent	Represents the rider's perspective	Gathers rider-side evidence, articulates the rider's claim based on company policy, and argues for rider-favorable outcomes.
Driver Advocate Agent	Represents the driver's perspective	Gathers driver-side evidence, articulates the driver's defense based on policy, and argues for driver-favorable outcomes.
Judge Agent	Acts as the impartial arbitrator	Weighs evidence from both advocates, applies company policy, issues a final ruling (refund, compensation, no action), and generates a natural language explanation of its reasoning.

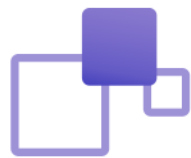
MVP Evidence Sources (Text & Structured Data):

- **GPS & Telemetry Data** — Route deviation analysis (comparing actual route vs. optimal route as data points), unexpected stops, trip duration vs. estimated time
- **Chat & Communication Logs** — Text analysis for sentiment, agreements, disagreements, and threats
- **Payment & Fare Data** — Fare breakdown validation, surge pricing disputes, promo code usage
- **Historical Behavior Profiles** — Rider and driver dispute history, rating patterns, account age

MVP Workflow:

- A dispute is filed (e.g., "The driver took a longer route and I was overcharged.")
- The Rider Advocate Agent and Driver Advocate Agent autonomously gather their respective evidence from the available data sources.
- Each advocate builds and presents its case, citing relevant company policy.
- The Judge Agent reviews both cases, applies policy, and issues a ruling with a confidence score and natural language reasoning summary.
- The system outputs the ruling, recommended action (e.g., partial refund of \$3.25), and explanation to both parties.

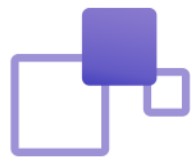




2. Stretch Goals (Bonus Points) — Advanced Agents & Multi-Modal Capabilities

The following are optional enhancements for teams that have completed the MVP and wish to demonstrate deeper technical capability. Implementing these will earn bonus points in the judging criteria.

Agent / Capability	Role	Why It's a Stretch Goal
Evidence Collection Agent	Autonomously orchestrates multi-modal data retrieval across all tools and APIs, feeding structured evidence to the advocate agents.	Adds orchestration complexity; requires tool-use and API integration design.
Fraud & Bad-Faith Detection Agent	Runs in parallel to assess whether either party exhibits patterns of dispute abuse, fake claims, or collusion. Feeds risk signals to the Judge Agent.	Requires behavioral modeling and risk scoring beyond the core dispute flow.
Policy & Precedent Agent	Maintains a dynamic knowledge base of company policies and past dispute rulings. Provides the Judge Agent with precedent-based recommendations for ruling consistency.	Requires building and querying a precedent knowledge base (e.g., RAG architecture).
Image Analysis (Multi-Modal)	Analyzes "mess" or damage photos for validity — checks if the photo is genuine, matches the trip timestamp, and is not AI-generated.	Introduces computer vision / multi-modal LLM capabilities — a significant technical leap.
Escalation Protocol	Confidence-scoring mechanism. When the Judge Agent's confidence falls below a threshold, it autonomously escalates to a human reviewer with a full case summary.	Adds conditional logic, threshold tuning, and human-in-the-loop design.
Learning Feedback Loop	When a human reviewer overrides the Judge Agent's ruling, the correction is captured and fed back into the Policy & Precedent Agent's knowledge base for continuous improvement.	Requires feedback ingestion and knowledge base updating — advanced system design.
SLA & Routing Manager	Prioritizes disputes by urgency and value. High-priority disputes (e.g., safety-related) are fast-tracked.	Adds a queue management and prioritization layer.



3. Strategic Guardrails for Participants

To ensure teams deliver a working prototype within the hackathon timeline, the following guardrails apply:

Scope Management

The three Core Agents handling text-based and structured evidence are the required deliverable. Teams will be evaluated primarily on this.

Advanced Agents, image processing, and multi-modal capabilities are stretch goals. Teams should only attempt these after the MVP is fully functional.

Teams are encouraged to mock or simulate external APIs and data sources where real APIs are unavailable. A sample dataset will be provided.

Technical Constraints

Teams may use any LLM or AI agent framework of their choice (e.g., LangChain, AutoGen, CrewAI, or custom implementations).

The system should be demoable — a working end-to-end flow for at least two dispute categories (e.g., route deviation + no-show charge) is expected by Demo Day.

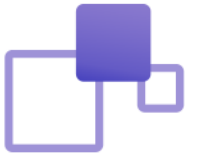
Inter-agent communication should be observable — judges should be able to see how agents exchange information and build their cases (e.g., via a visible communication log or UI).

Dispute Categories (Sample)

Teams should ensure their system can handle at least two of the following dispute types:

	Dispute Type	Example
1	Route Deviation	"Driver took a longer route and I was overcharged."
2	No-Show Charge	"Driver didn't show up but I was charged a cancellation fee."
3	Property Damage / Mess	"Rider spilled drinks in the car and driver is claiming cleaning fees." (requires image analysis — stretch goal)
4	Safety Incident	"Driver behaved inappropriately during the trip."





The Solution Should Include

- **Working Prototype** – An end-to-end solution capable of handling at least two dispute categories.
- **Architecture Diagram** – A clear visual representation of the multi-agent system and key components.
- **Demo Walkthrough** – A live demonstration showing how a dispute is processed and resolved autonomously.
- **Source Code** – Complete source code submitted through a GitHub repository.

Note: *The features listed above are provided as guidance only. Participants are strongly encouraged to explore alternative approaches that meaningfully address the problem statement.*